

ARTICLE INFO

Keywords:

E-Voting

ElectionGuard

Homomorphic Encryption

end-to-End Verifiability

Microservice Architecture

Mobile Security

ABSTRACT

Dijital seçim sistemleri, seçmen gizliliğini korurken sonuçların doğrulanabilirliğini sağlamak konusunda kritik bir güven sorunuyla karşı karşıyadır. Bu çalışma, Microsoft tarafından geliştirilen ElectionGuard 2.1.0 protokolünü temel alan, çok katmanlı mikroservis mimarisine sahip Pallot/Vocket adlı uçtan uca doğrulanabilir (E2E-V) bir mobil oylama sistemini sunmaktadır. Sistem, homomorfik şifreleme ve sıfır-bilgi kanıtları kullanarak seçmen gizliliğini garanti altına alırken her seçmenin oyunun sandığa doğru şekilde yansıtıldığını matematiksel olarak doğrulayabilmesine olanak tanır.

1. Introduction

Elections, the cornerstone of democratic processes, require a reliable, transparent, and manipulation-resistant infrastructure in modern societies. Traditional paper-based voting systems have represented a tangible expression of this trust for centuries. However, the increasing mobile connectivity and remote service demands of societies due to digital transformation are creating strong pressure to move election processes to electronic environments. Electronic voting systems offer voters independence from time and place, as well as significantly reducing counting errors and bureaucratic costs [1]. However, these advantages also bring to the forefront a number of critical security issues that are either absent or much more limited in traditional election systems.

The most fundamental challenge in designing electronic voting systems is having to simultaneously meet two fundamentally contradictory requirements: confidentiality and verifiability. On the one hand, there is the principle of voter secrecy, meaning that a voter's choice must under no circumstances be learned by third parties; on the other hand, there is the necessity of transparency, which allows the voter to independently verify whether their vote has been actually recorded and whether the counting has been done correctly. Meeting these two requirements simultaneously constitutes a profound theoretical problem that is not easily achieved in classical information security paradigms and can only be solved with advanced cryptographic protocols [2].

End-to-end verifiability (E2E-V) is the most comprehensive cryptographic framework developed to solve this problem. E2E-V protocols aim to provide two fundamental properties simultaneously: individual verifiability, meaning that a voter can independently verify that their vote has been recorded correctly; and universal verifiability, meaning that any observer can verify that all recorded votes have been correctly counted. The core cryptographic tools of this framework include homomorphic encryption, zero-knowledge proofs (ZKP), and threshold decryption protocols [3, 4].

Homomorphic encryption is a special type of encryption that allows votes to be collected while they are encrypted [2]. Individual voter preferences are never processed in plaintext; only the combined total is deciphered by parties possessing a sufficient number of keys. The ElGamal encryption system [5] is one of the leading cryptographic structures widely used for this purpose, offering a homomorphic property through summation. Zero-knowledge proofs [6] are protocols that can prove a voter has actually cast a valid vote without explicitly stating it. The integrated use of these two mechanisms makes it possible to guarantee the integrity of encrypted votes while keeping their contents confidential [7].

One of the most important initiatives in bringing this theoretical framework to life as a practically applicable electoral system is ElectionGuard. Announced in 2019 as part of Microsoft's Defending Democracy program, ElectionGuard is designed as an open-source software development kit (SDK) that enables E2E-V elections [8]. The system's fundamental principle is to create the cryptographic layer independently of existing voting infrastructures, thus allowing any electoral system manufacturer to easily integrate E2E-V support into their platform. This decoupling

*Corresponding Author. E-mail: samet.tonyali@gumushane.edu.tr

 mf.bilgin0@hotmail.com (M.F. Bilgin); samet.tonyali@gumushane.edu.tr (S. Tonyali)

ORCID(s):

approach allows security experts to independently design and develop cryptographic components, enabling implementation without the need to restructure the existing electoral infrastructure [9].

ElectionGuard's operation is built on four core components:

1. **Key Ceremony:** Before the election, trusted parties called guardians perform a key generation ceremony. Each guardian generates its own partial private-public key pair, and the guardians cooperate to create a common public key (Joint Public Key) used to encrypt the votes.
2. **Vote Encryption:** During the voting process, each voter's choice is encrypted using the ElGamal homomorphic encryption method, and Chaum–Pedersen zero-knowledge proofs are generated, demonstrating the well-formed nature of the ballot.
3. **Homomorphic Combination:** A combined encrypted count is obtained by homomorphically combining all encrypted ballots.
4. **Threshold Decryption:** The minimum threshold number (quorum) of guardians provides the partial decryption share, revealing the final election results.

This architecture ensures that no single administrator or group of guardians can monopolize the entire decryption process; trust is maintained in a distributed structure [10].

The verification code returned to the voter forms the core tool for individual verifiability. After casting their vote, voters can verify that their vote has been recorded without alteration by querying this code on the bulletin board. The hash-chaining of numerous votes makes it possible to detect any secret additions or deletions of votes from the bulletin board. ElectionGuard's key innovation is its separation of cryptographic tools from the core mechanics and user interface of the election system; this allows security experts to focus on the cryptography layer while election system manufacturers maintain the familiar infrastructure [9]. Owing to this, ElectionGuard has been piloted in real elections in the US states of Wisconsin, Idaho, Maryland, and Utah [11].

Voting via mobile devices adds a new layer of complexity to the implementation of E2E-V protocols. The computational capacity and reliable hardware support of smartphones can be limited compared to desktop or dedicated election equipment. This severely restricts the execution of large cryptographic operations directly on the mobile client [12]. In addition, the reliability of the mobile network connection, vulnerabilities arising from network latency, and varying hardware security capabilities per device present unique problems that require investigation in terms of both individual performance and system-level scalability.

The vast majority of existing E2E-V systems target corporate or web-centric environments and fail to adequately address mobile user scenarios. While Helios [13], a web-based system, has been successfully used in certain qualified elections such as the IACR (The International Association for Cryptologic Research) elections and university elections, it falls short for large-scale and mobile-focused applications due to a lack of native mobile integration and an integrated asynchronous auditing mechanism. Belenios [14], built on Helios' cryptographic architecture, offers strong verifiability features, particularly against bulletin board manipulation; however, it also lacks a native mobile architecture. Blockchain-based solutions, while offering advantages such as decentralization and traceability, are limited for large-scale election applications due to computational intensity and scalability issues [15].

The main gap observed in the current literature is the lack of an integrated system that combines E2E-V cryptographic guarantees with a real-time auditing infrastructure and a cloud-native microservice architecture, treating native mobile access as the primary channel. Existing approaches, where the cryptographic engine, business logic services, and auditing layer are housed within the same monolithic or semi-centralized structure, exhibit significant limitations in terms of scalability, service isolation, and cryptographic computation efficiency.

Driven by this motivation, the Pallot/Vocket system integrates the ElectionGuard SDK into a multi-layered microservice architecture, offering both E2E-V-enabled and mobile-first voting solutions. The main contributions of this work can be summarized as follows:

- (i) Design and implementation of a hybrid cryptographic protocol integrating RSA-OAEP for transit security, ElGamal homomorphic encryption for persistent vote recording, and Chaum-Pedersen ZKP generation,
- (ii) A stateless cryptographic engine architecture that directly supports horizontal scaling,
- (iii) An asynchronous moderation bulletin board service independent of the main voting stream, that updates in real-time via the RabbitMQ message queue,

- (iv) Full integration of the React Native-based mobile client with the E2E-V protocol and implementation of an individual verification code-based flow.

The remainder of this article is structured as follows: Section 2 reviews the relevant studies. Section 3 details the system architecture and cryptographic protocol. Section 4 details the implementation, and Section 5 addresses the security analysis. Section 6 presents the performance evaluation, and finally, Section 7 discusses the conclusions.

2. Related Work

Academic studies in the field of electronic voting have gained significant depth over the last two decades, both in terms of cryptographic protocol design and system architecture. This section examines the main studies directly related to the design decisions of the Pallot/Vocket system. The relevant literature can generally be categorized under four main headings: web-based E2E-V systems, blockchain-based approaches, mobile voting studies, and cryptographic protocol developments.

2.1. Web-Based E2E-V Voting Systems

One of the most influential works in the field of E2E-V voting systems is the Helios system, developed by Ben Adida in 2008 [13]. Helios pioneered the introduction of cryptographically verifiable elections to a wide audience with its fully web-based, open-audit structure accessible to everyone. Using ElGamal-based homomorphic encryption, the system allows voters to submit their encrypted ballots via a browser and publish the ballots on a publicly accessible bulletin board. It has been successfully implemented in IACR, ACM, and various university elections. However, due to its centralized structure, Helios cannot offer strong guarantees against bulletin board manipulation; it also lacks local mobile integration and real-time auditing mechanisms. Bernhard et al. [2] made a significant theoretical contribution by documenting the weaknesses in Helios' implementation of zero-knowledge proofs based on the Fiat-Shamir heuristic.

The Belenios system, developed by Cortier et al. [14], represents an advanced E2E-V architecture designed to address the shortcomings of Helios. Belenios prevents the bulletin board from adding fraudulent votes or altering valid ballots by adopting a distributed structure where voter credentials are managed by a separate identity authority. The system can cryptographically prove the validity of ballots using disjunctive Chaum–Pedersen proofs and strengthens universal verifiability assurances. Cortier et al. [16] validated Belenios' privacy and verifiability features with machine-controlled proofs and contributed to a formal security analysis of the protocol. However, Belenios maintains a web-centric server architecture and exhibits significant limitations in terms of native mobile integration and cloud-native scalability.

In contrast with Helios and Belenios, ElectionGuard [8, 9] is not positioned as a standalone, complete voting system; instead, it offers a modular cryptographic toolkit to add E2E-V support to existing voting systems. As detailed in the ElectionGuard v2.1.0 Design Specification [10], the system integrates guardian-based distributed key generation, homomorphic vote aggregation, Chaum–Pedersen zero-knowledge proofs, and hash-chain based verification codes. The system's usability in real elections has been validated through pilot applications in Fulton, Wisconsin (2020) and Franklin County, Idaho (2022) [11]. Pallot/Vocket adopts ElectionGuard as its core cryptographic engine, extending it with a cloud-native microservice architecture and native mobile client integration.

2.2. Traditional E2E-V System Designs

STAR-Vote [17], designed by Bell et al. for the Texas elections, is a hybrid model that integrates a paper-based audit trail with homomorphic encryption. Offering both physical security and mathematical proofs under a single roof, this system is a reference for Risk-Limiting Audit (RLA) applications. ElectionGuard's RLA support is partly inspired by this STAR-Vote approach. Pallot/Vocket, on the other hand, follows a digital-first architecture that completely eliminates the need for physical auditing.

Civitas [18], proposed by Clarkson, Chong, and Myers, offers one of the most comprehensive theoretical contributions to the literature on coercion resistance. By using mix-net technology and dummy credentials to prevent votes cast under duress from being counted, Civitas increases technical complexity due to its high security level and makes practical optimization for mobile use difficult. Pallot/Vocket does not fully resolve coercion resistance in its current design. This leaves it an open research question for future studies.

Selene [19], developed by Ryan, Rønne, and Iovino, offers voters an innovative tracker code mechanism to verify that their votes have been counted correctly. Selene's main contribution is that it incorporates cryptographic measures

to prevent vote buying while ensuring individual verifiability. The verification code mechanism in Pallot/Vocket is inspired by Selene's approach and guarantees complete receipt-freeness.

Prêt à Voter [20], proposed by Ryan et al., is designed as a hybrid system that offers verifiability to voters in a physical paper environment using a randomized candidate order technique. Unique in its integration of cryptographic control traces with paper documents, this system formed the basis for the standard formulation of the E2E-V concept.

The Norwegian Internet Voting System [21], analyzed by Gjøsteen, represents one of the rare examples of E2E-V implemented at the national level. The system provided individual verifiability through verification codes sent to voters via email/SMS. However, it was discontinued in 2014 due to resistance to enforcement and scalability issues. This experience offers important lessons about the complexity of real-world election systems and the critical importance of trust models in mobile/Internet environments.

2.3. Blockchain-Based E-Voting Approaches

Blockchain technology is attracting increasing interest in the field of electronic voting due to its decentralized trust model and immutable record structure. Panja and Roy [22] formally analyzed the security guarantees of this combination by proposing an E2E-V e-voting system that combines blockchain and cloud servers. Their work highlighted the contributions of blockchain in terms of integrity and traceability, while drawing attention to latency and scalability issues, particularly in large-scale elections. Zaghoul et al. [15] presented D-BAME, an anonymous and distributed blockchain-based mobile e-voting system. Their work demonstrated that it is possible to balance anonymity and traceability requirements on mobile devices. However, both approaches lack cryptographic E2E-V guarantees at the ElectionGuard level and remain limited in terms of proof-building and verification mechanisms.

2.4. Mobile E-Voting Studies

The transfer of election systems to smartphones has attracted the attention of researchers due to both practical and security considerations. Peter et al. [12] developed a mobile e-voting application integrating a three-stage security verification mechanism to prevent phishing attacks. While the system offers tangible contributions in terms of strengthening access security, it does not address the issue of cryptographic verifiability. Devi and Bansal [23] comprehensively compared various e-voting systems in terms of their security features and revealed that E2E-V proofs are not systematically addressed in mobile-focused studies. This finding points to the importance of integrated approaches to bridging the gap between mobile user convenience and cryptographic assurances.

2.5. Cryptographic Protocol and Infrastructure Developments

Several studies that have made significant contributions to the theoretical foundation of election cryptography should also be considered within the relevant literature. Chaum and Pedersen [7] laid out the fundamental principles of homomorphic encryption and observer-based privacy models. The Chaum-Pedersen protocol is widely used today as a standard ZKP structure for proving the validity of ballots in ElGamal-based voting systems. Farzaliyev et al. [24] investigated lattice-based mix-nets for the post-quantum era and showed that systems based on standard ElGamal encryption are vulnerable to quantum computing. This finding points to an important research direction that should be addressed in the future for the current ElGamal implementation of Pallot/Vocket.

Benaloh et al. [11], by documenting experiences gained from implementing ElectionGuard technology in collaboration with manufacturers operating in real election environments, addressed the institutional, legislative, and usability barriers encountered in the practical adoption of E2E-V systems from a broad perspective. This work emphasizes that providing cryptographic guarantees alone is insufficient and that legal compliance requirements also play a critical role. Daraghmi et al. [25], in their research on microservice and IoT-based e-voting architectures, demonstrated the advantages of separating service layers in distributed voting systems in terms of security and scalability. However, they failed to provide a system architecture that included cryptographic ZKP proofs and real-time auditing mechanisms.

The studies reviewed show that significant progress has been made in the field of e-voting. Helios and Belenios have proven that E2E-V cryptographic protocols can be implemented in a web environment. ElectionGuard has taken a paradigmatic step in this area by offering a modular and open-source cryptographic toolkit. Blockchain-based approaches have produced unique solutions regarding decentralization. However, it is clear that a system with a cloud-native microservice architecture that integrates E2E-V assurances with a real-time asynchronous audit infrastructure, treats native mobile access as the primary channel, and houses the cryptographic engine in isolation from business logic services is lacking in the literature. Pallot/Vocket is positioned as a unique contribution aiming to fill precisely this gap.

Table 1

Sistem Katmanları ve Sorumlulukları

Katman	Bileşen	Sorumluluk
Client	React Native / Expo	Oy şifreleme, ZKP, transit şifreleme
Edge	OpenResty (Nginx + Lua)	TLS sonlandırma, JWT doğrulama, hız sınırı
Application	5 × Spring Boot servisi	İş mantığı, seçim yaşam döngüsü
Crypto	Python gRPC + Kotlin POC	ElectionGuard işlemleri
Persistence	PostgreSQL, MongoDB, Redis	İlişkisel veri, denetim defteri, ön bellek
Observability	Prometheus + Loki + Grafana	Metrik, log, gösterge paneli

3. System Architecture

Pallot/Vocket, ElectionGuard 2.1.0 protokolünü temel alan, katmanlı ve mikroservis tabanlı bir uçtan uca doğrulanabilir mobil oylama platformudur. Sistemin tasarım felsefesi, kriptografik bileşenleri iş mantığından, denetim altyapısını ise oylama akışından izole etmek üzerine kuruludur. Bu sayede her katman bağımsız olarak ölçeklenebilir, güncellenebilir ve denetlenebilir hâle gelmektedir. Sistemin temel özelliği, oy şifrelemenin tamamen istemci tarafında gerçekleştirilmesidir; sunucu tarafı hiçbir noktada seçmenin tercihini düz metin olarak görmez.

3.1. High-Level Architecture

Sistem, birbirinden işlevsel olarak ayrılmış altı katmandan oluşmaktadır. Tablo 1 bu katmanları, bileşenlerini ve birincil sorumluluklarını özetlemektedir.

Şekil 1, katmanlar arasındaki veri akışını ve protokol sınırlarını göstermektedir.

Mobil istemci, HTTPS/TLS 1.2/1.3 üzerinden yalnızca OpenResty API Gateway'ye bağlanır. Her istekle birlikte X-Correlation-ID (UUID v4) ve X-Device-ID başlıkları taşınır; bu başlıklar, dağıtık izlenebilirliği sağlamak amacıyla tüm servislerin günlük kayıtlarına eklenir. Gateway, public JWT'yi doğruladıktan sonra kısa ömürlü (TTL 30 s) bir internal JWT üretir ve bunu X-Internal-Auth başlığıyla uygulama servislerine iletir. Böylece bir servisin ele geçirilmesi durumunda saldırgan, diğer servislere geçerli kimlik bilgisi üretemez.

3.2. Application Layer: Microservices

Uygulama katmanı, her biri kendi PostgreSQL veritabanına sahip ve birbirinin veri depolama alanına doğrudan erişemeyen beş Spring Boot servisinden oluşmaktadır. Tüm servisler Java 21, JPA/Hibernate, Spring Security ve Actuator (/actuator/health, /actuator/prometheus) kullanır; durumsuz (stateless) ve yatay ölçeklenebilir biçimde tasarlanmıştır.

3.2.1. user-service (8080)

Kimlik doğrulama, kullanıcı profili yönetimi, iki faktörlü kimlik doğrulama (2FA/TOTP), parola sıfırlama ve denetim günlüğü işlevlerini üstlenir. Spring Boot 3.5.5, jjwt 0.12.5 ve Bucket4j kütüphanelerini kullanan servis; Redis'i token/oturum ön belleği, AWS S3'ü avatar depolama ve AWS SES'i e-posta gönderimi için kullanır. Başarılı kayıt ve parola sıfırlama gibi olaylar RabbitMQ email.exchange (Direct) kuyruğuna bırakılır; servis bu kuyruğu aynı zamanda kendi tüketerek e-posta gönderimini tetikler. Kritik kullanıcı eylemleri, X-Correlation-ID ile ilişkilendirilmiş audit_logs tablosuna kaydedilir.

3.2.2. community-service (8083)

Topluluk oluşturma, üyelik yönetimi (OWNER / MODERATOR / MEMBER), davet kodu üretimi ve istatistik işlevlerini sağlar. Servis, InternalCommunityController aracılığıyla yalnızca gateway tarafından imzalanmış internal JWT'ye yanıt veren dahili uç noktalar sunar. election-service, bir seçmen katılım hakediyor mu diye doğrulamak amacıyla bu uç noktaları sorgular.

3.2.3. election-service (8082)

Sistemin merkezi orkestratördür. Spring Boot 3.4.3, gRPC 1.62.2 ve Protobuf 3.25.3 kullanan bu servis, kriptografi motoruyla gRPC, diğer servislerle REST üzerinden iletişim kurar. Seçim yaşam döngüsü, Şekil 2'de gösterilen durum makinesiyle yönetilir.

Election varlığı (entity); electionGuardContext (JSON), electionManifest, electionPublicKey (birleşik ElGamal anahtarı) ve minGuardiansThreshold (Q) alanlarını barındırır. Vote varlığı ise yalnızca encryptedBallot (CiphertextBallot JSON), zkpProof, ballotHash (SHA-256) ve trackingCode saklayacak şekilde tasarlanmıştır. Çift oy önleme, anonim bir VoteNullifier kaydı aracılığıyla gerçekleştirilir; bu yapı, seçmen kimliğini oy içeriğiyle ilişkilendirmeksizin yinelenmiş girişimleri engeller.

3.2.4. bulletin-board-service (8085)

Şeffaf ve kurcalamaya karşı dayanıklı (tamper-evident) kamuya açık denetim defterini yönetir. Her kayıt türü (BALLOT, TALLY, METADATA) için hash zinciri şu formülle hesaplanır:

$$h_i = \text{SHA-256}(\text{electionId} \parallel \text{recordType} \parallel \text{trackingCode} \parallel \text{ballotHash} \parallel h_{i-1} \parallel \text{payload}) \quad (1)$$

Bu yapı, bülten panosuna gizlice ekleme ya da silme yapılmasını kriptografik olarak tespit edilebilir kılar. Herhangi bir bağımsız doğrulayıcı, zinciri başından sonuna tarayarak tutarsızlıkları saptayabilir. Depolama için MongoDB tercih edilmiştir; ekleme-yoğun (append-only) ve belge merkezli erişim kalıbı, hash zincirinin doğal modeli olan bu veritabanında verimli biçimde desteklenmektedir.

3.2.5. notification-service (8084)

Durumsuz bir mesaj tüketicisi olarak çalışır. RabbitMQ notification.exchange (Topic) üzerindeki notification.election.queue'yu dinler. Guardian ataması, seçim başlangıç/bitiş ve oy gönderim olaylarını tüketerek kullanıcının tercihinine göre (NotificationPreference) Expo Push API üzerinden mobil bildirim ya da AWS SES üzerinden HTML e-posta gönderir. Başarısız mesajlar Dead Letter Queue'ya (notification.election.dlq) yönlendirilir; böylece geçici ağ hatalarında mesaj kaybı önlenir.

3.3. Cryptographic Engine

Kriptografik hesaplamaların iş mantığından izole edilmesi, Pallot/Vocket mimarisinin temel tasarım ilkelerinden birini oluşturmaktadır. Bu tasarım kararının temel gerekçesi, ElectionGuard işlemlerinin hesaplama yoğunluğu ve bağımlılık ağacının geniş kapsamıdır; kriptografi motorunu bağımsız tutmak, her iki tarafın da bağımsız olarak ölçeklenmesine ve güncellenmesine imkân tanır.

3.3.1. Python Crypto Engine (Üretim)

Python 3.10 üzerinde çalışan ve port 50051'de dinleyen servis, ElectionGuard 1.4.0 Python SDK'sını GitHub deposunun v1.4.0 etiketinden kurar; PyPI'deki sürüm bazı üst-seviye yardımcı modülleri içermediğinden doğrudan kaynak deposu kullanılmaktadır. Servis beş gRPC prosedürünü gerçekleştirir: SetupElection, ValidateBallot, CreateJointKey, TallyElection ve DecryptWithShares.

Servisin en önemli mimari özelliği **durumsuz** olmasıdır: her gRPC çağrısı, manifest, şifreleme bağlamı, oy pusulası ve guardian kayıtları dahil tüm bağlamı kendi payload'ında taşır; servis, bellek veya disk üzerinde hiçbir veri saklamaz. Bu tasarım sayesinde crypto engine podları, bir yük dengeleyicinin arkasında kolaylıkla yatay olarak ölçeklendirilebilir.

ElectionGuard Hash Zinciri. ElectionGuard spesifikasyonu, şifreleme bağlamını türetmek için hiyerarşik bir karma (hash) zinciri tanımlar. SDK'nın v1.4.0 sürümünde bazı üst-seviye yardımcılar eksik olduğundan bu zincir, crypto_servicer.py içinde şekil 3'de gösterildiği gibi manuel olarak inşa edilmektedir:

$$H_{\text{base}} = \text{hash_elems}(N, Q, H_{\text{manifest}}) \quad (2)$$

$$H_{\text{ext}} = \text{hash_elems}(H_{\text{base}}, H_{\text{commit}}) \quad (3)$$

burada N toplam guardian sayısını, Q quorum eşliğini, $H_{\text{manifest}} = \text{Manifest.crypto_hash}()$ seçim yapısının özeti ve $H_{\text{commit}} = \text{hash_elems}(\text{tüm guardian taahhütleri})$ değerini göstermektedir. Elde edilen H_{ext} (crypto_extended_base) hem oy pusulası şifreleme hem de Chaum–Pedersen ZKP doğrulamasında referans noktası olarak kullanılır.

Guardian Anahtar Töreni. Sistem iki guardian ceremony modunu destekler. Geliştirme ve test ortamlarında kullanılan Mod 1, tüm guardian anahtar çiftlerini tek bir süreçte merkezi olarak oluşturur; üretim ortamında kullanılmaz. Üretim için tasarlanmış Mod 2’de her guardian, private anahtarını kendi mobil cihazında üretir ve sunucuya yalnızca açık anahtarını (*public key*), polinom taahhütlerini (*commitments*) ve Schnorr katsayı kanıtlarını iletir. Private key hiçbir zaman ağ üzerinden geçmez. `distributed_ceremony.create_joint_key()` fonksiyonu her guardian’ın Schnorr kanıtını `is_valid(commit_key)` çağrısıyla doğrular, ardından joint key’i $K_{\text{joint}} = \prod_{i=1}^N K_i \pmod{p}$ formülüyle hesaplar. Sayım aşamasında her guardian kendi kısmi şifre çözüm payını $d_i = T^{s_i} \pmod{p}$ biçiminde hesaplar; en az Q paydan Lagrange interpolasyonu ile düz metin sayım elde edilir.

Chaum–Pedersen ZKP. Her seçimin (α, β) ElGamal şifreli metni için bir aralık kanıtı üretilir. Bu kanıt, doğrulayıcının düz metin tercih içeriğini bilmeksizin şu iki iddiayı teyit etmesine olanak tanır: (1) şifreli seçim $\{0, 1\}$ kümesinde geçerli bir değer içermektedir; (2) seçim sayı sınırı aşılmamıştır (örneğin tek tercihli bir yarışmada en fazla bir aday seçilmiştir). ElectionGuard SDK’sı, tüm kanıtları toplu olarak şu çağrıyla doğrular:

```
ballot.is_valid_encryption(
    manifest_hash, elgamal_public_key,
    crypto_extended_base_hash)
```

Şifreli oy üzerinde herhangi bir değişiklik yapılması bu kanıtı geçersiz kılar ve böylece kötü niyetli müdahaleler matematiksel olarak tespit edilebilir hâle gelir [10].

3.3.2. Kotlin Crypto Engine (POC)

Çapraz gerçekleştirim doğrulaması amacıyla, aynı gRPC sözleşmesini `votingworks/electionguard-kotlin-multiplatform` üzerinde uygulayan ikinci bir servis geliştirilmiştir. Spring Boot 3.2.5 ve Kotlin 1.9.22 kullanan bu servis port 50052’de dinler; `SetupElection` ve `ValidateBallot` prosedürleri işlevseldir, `TallyElection`, `CreateJointKey` ve `DecryptWithShare` ise henüz uygulanmamıştır (UNIMPLEMENTED). Üretim trafiği Python motorundan geçerken Kotlin POC, paralel akışları dinleyerek tutarlılık testlerinde kullanılmaktadır. Bu yapının uzun vadeli amacı, aynı KMP kitaplığını hem sunucu hem de mobil native modül olarak kullanarak tek tip bir kriptografi yığını oluşturmaktır.

3.4. Mobile Client

İstemci katmanı, React Native 0.83.6 ve Expo 55.0.23 çerçevesinde TypeScript ile geliştirilmiştir. Oy şifreleme, `node-forge` 1.4.0 kütüphanesi ve Android’de Kotlin native modülü olarak entegre edilmiş `ElectionGuardClient` aracılığıyla cihaz üzerinde gerçekleştirilir; iOS için aynı KMP modülünün derlenmesi üçüncü aşama çalışma olarak planlanmaktadır. JWT erişim tokeni ve guardian secret anahtarı, işletim sistemi tarafından şifrelenen `expo-secure-store`’da saklanır; bu değerler düz metin olarak uygulama belleğinde asla tutulmaz.

Kimlik Doğrulama ve Token Yönetimi. `AuthContext`, erişim tokeni (TTL 900 s) ve yenileme tokeni çiftini `SecureStore`’da yönetir. `axios` istek kesicisi (*interceptor*) her isteğe `X-Correlation-ID` (UUID v4), `X-Device-ID`, `X-App-Version` ve `X-Platform` başlıklarını ekler. Yanıt kesicisi 401 hatalarını `failedQueue` mekanizmasıyla sıraya alarak eş zamanlı 401 durumlarında yalnızca tek bir yenileme isteği gönderilmesini sağlar; yenileme başarısız olursa `DeviceEventEmitter` üzerinden oturum kapatma olayı yayınlanır.

Oy Verme Akışı. Şekil 4, uçtan uca oy verme sıra diyagramını göstermektedir.

İstemci önce `GET /elections/{id}/encryption-params` uç noktasından `electionGuardContext`, `electionManifest` ve `jointPublicKey` değerlerini alır. Ardından `POST /votes/token` ile bir `VoteToken` (uygunluk belgesi) edinir. Seçmen tercihini yerel olarak oluşturup `ElectionGuardClient.encryptBallot()` çağrısını tetikler; bu çağrı `CiphertextBallot`, `zkpProof` ve bir onay kodu (*tracking code*) döndürür. Öte yandan istemci, çift oy saldırılarını önlemek amacıyla `nullifierHash = SHA-256(token || ballot_structure)` formülüyle anonim bir nullifier üretir. Son adımda `CiphertextBallot`, hibrit RSA-OAEP+AES-GCM ile şifrelenerek sunucuya iletilir. Sunucu düz metin tercihi hiçbir zaman görmez; yalnızca şifreli oy ve ZKP kanıtı işlenir.

Guardian Modülü. Guardian rolündeki kullanıcı, `guardian_crypto.ts` modülü aracılığıyla 256-bit bir s secret değeri üretir ve `SecureStore`’a kaydeder. Açık anahtar $K = g^s \pmod{p}$ (4096-bit ElectionGuard temel grubu), Schnorr kanıtı $\{h, c, v\}$ ve $c = \text{SHA-256}(G\|K\|h) \pmod{q}$, $v = (k + c \cdot s) \pmod{q}$ biçiminde hesaplanır. Bu değerler `POST`

Table 2

Senkron Servisler Arası İletişim Kanalları

Kaynak	Hedef	Protokol	Not
Mobile	Gateway	HTTPS/TLS	Public JWT
Gateway	Servisler	HTTP/JSON	Internal JWT (30 s TTL)
election-svc	community-svc	HTTP/JSON	Internal JWT
election-svc	user-svc	HTTP/JSON	Internal JWT
election-svc	bulletin-board	HTTP/JSON	Internal JWT
election-svc	crypto-engine	gRPC/Protobuf	30 s / 120 s timeout
election-svc	crypto-kotlin	gRPC/Protobuf	Paralel POC

/guardians/{u}/keys ile sunucuya iletilir; private key cihazı hiçbir zaman terk etmez. Sayım aşamasında guardian, $\text{decryptionShare} = T^s \bmod p$ 'yi hesaplayıp gönderir; bu işlemde de private key yalnızca anlık bellekte bulunur ve ağa asla çıkmaz.

3.5. API Gateway

OpenResty (Nginx + LuaJIT), sistemin dış sınırında tek giriş noktası işlevi görür. Lua betikleri, JWT doğrulama, hız sınırlama ve korelasyon kimliği yönetimi gibi işlevleri gerçekleştirir.

Yönlendirme ve Hız Sınırlaması. POST /v1/auth/* uç noktaları için kimlik doğrulama gerekmeksizin IP başına dakikada 10 istek, korumalı uç noktalar için ise kullanıcı başına dakikada 60 istek sınırı uygulanmaktadır. Sınır aşıldığında 429 Too Many Requests ile birlikte Retry-After başlığı döndürülür.

Internal JWT. Gateway, gelen public JWT'yi doğruladıktan sonra kısa ömürlü (TTL 30 s) ve kapsamı sınırlı bir internal JWT imzalar. Bu token X-Internal-Auth başlığıyla uygulama servislerine iletilir; servisler yalnızca bu token'ı doğrular ve X-User-Id, X-Platform-Role değerlerine güvenir. Bu mekanizma, bir servisin ele geçirilmesi durumunda yatay hareketin önüne geçer.

TLS ve Güvenlik Başlıkları. Dış bağlantılar TLS 1.2 ve TLS 1.3 ile korunur. Let's Encrypt sertifikaları 12 saatte bir otomatik olarak yenilenir. HSTS (max-age=31536000), X-Frame-Options: SAMEORIGIN, X-Content-Type-Options: nosniff ve X-XSS-Protection başlıkları tüm yanıtlara eklenir. Büyük ElectionGuard manifest ve kanıt JSON'larını karşılayabilmek için proxy_buffer_size 128k, proxy_buffers 4 × 256k ve proxy_read_timeout 120s olarak yapılandırılmıştır.

3.6. Inter-Service Communication and Asynchronous Audit

Servisler arası iletişim, senkron ve asenkron kanalların bilinçli kombinasyonuna dayanmaktadır.

Senkron İletişim. Şifreli oy doğrulama için election-service'ten crypto-engine'e yapılan gRPC çağrısında 30 saniyelik zaman aşımı uygulanır; sayım (tally) işlemi için bu süre 120 saniyeye çıkar. İkili (binary) Protobuf serileştirme ve HTTP/2 çoklama (multiplexing), büyük ElectionGuard manifestlerinin düşük gecikmeyle iletilmesini sağlar. Tablo 2 tüm senkron iletişim kanallarını özetlemektedir.

Asenkron Denetim Kuyruğu. Oy gönderiminin kullanıcı deneyimini sekteye uğratmaması için denetim defterine yazım işlemi asenkron gerçekleştirilmektedir. election-service, oyu PostgreSQL'e kaydeder ve kullanıcıya anında HTTP 200 döndürür; aynı anda notification.exchange (Topic) kuyruğuna bir olay yayımlar. bulletin-board-service bu olayı kendi temposunda tüketerek MongoDB'deki hash zincirini günceller.

RabbitMQ'da iki exchange tanımlıdır: kullanıcı e-postası için email.exchange (Direct) ve seçim bildirimleri için notification.exchange (Topic). Her exchange'e karşılık gelen Dead Letter Queue (DLQ) mevcuttur; bu yapı, geçici servis hatalarında mesaj kaybını önler. Şekil 5 asenkron olay akışını göstermektedir.

3.7. Data Persistence

Veri kalıcılığı katmanı, mikroservis sınırlarını veri seviyesinde de koruyacak biçimde tasarlanmıştır.

Table 3

Backend'in Sakladığı ve Saklamadığı Veriler

Veri	Saklanır mı?	Notlar
Düz metin oy tercihi	Hayır	Mobilden çıkmaz
encryptedBallot	Evet	E2E-V denetim izi
zkpProof, ballotHash, trackingCode	Evet	Doğrulama, bütünlük, makbuz
Guardian private key	Hayır	Yalnızca SecureStore'da
decryptionShare	Evet	Tek başına deşifre edemez; <i>Q</i> adet gerekir
tallyProof, tallyResults	Evet	ElectionGuard sayım kanıtı

PostgreSQL 15 — Dört İzole Veritabanı. Her servis yalnızca kendi veritabanına erişir: userdb (kimlik doğrulama ve denetim), communitydb (topluluk ve üyelik), electiondb (seçim, oy, guardian, sonuç) ve bulletindb (bülten panosu alternatif PostgreSQL arka ucu). Bu yapıtım, bir servis güvenliği ihlali durumunda diğer servislerin veri alanlarının korunmasını garantiler. electiondb, V1'den V11'e kadar 11 Flyway migrasyonu içerir. En kritik değişiklik, E2E-V uyumluluğunu sağlamak amacıyla V11 ile uygulanan votes.candidate_id sütununun kaldırılması ve vote_nullifiers tablosunun eklenmesidir.

MongoDB — Bülten Panosu Defteri. Bülten panosu kayıtları, _id, electionId, recordType, ballotHash, previousHash, recordHash ve payload alanlarını içeren belge yapısında saklanır. recordHash değeri Denklem (1)'deki formülle hesaplanır.

Redis 7 ve Backend'in Gizlilik Garantisi. Redis, token önbelleği ve hız sınırı sayaçları için kullanılmaktadır. Tablo 3, sistemin hangi verileri sakladığını ve hangilerini asla kayıt altına almadığını özetlemektedir.

3.8. Infrastructure and Observability

Üretim ortamında 16 Docker container tek bir köprü ağı (172.19.0.0/16) üzerinde çalışmaktadır. Bellek bütçesi yaklaşık 2.5 GB olarak planlanmış ve 4 GB RAM'e sahip bir sunucuya sığdırılmıştır: Gateway 256 MB, user-service 512 MB, election-service ve community-service 384 MB, bulletin-board 256 MB ve crypto-engine 512 MB.

Şekil 6 container topolojisini ve bağımlılıklarını göstermektedir.

PostgreSQL için cron tabanlı yedekleme servisi günlük pg_dump | gzip alır ve 7 gün saklar. Servis sağlığı, Prometheus tarafından her 30 saniyede bir /actuator/prometheus uç noktasından çekilen Micrometer metrikleriyle izlenir. Loki + Promtail tüm container günlüklerini Docker socketinden toplar; her satır timestamp, level, service, correlation_id ve user_id alanlarını içeren JSON yapısındadır. Bu sayede tek bir oy verme işleminin tüm servislerdeki izi, yalnızca X-Correlation-ID değeriyle Grafana üzerinden sorgulanabilmektedir.

References

- [1] J. P. Gibson, R. Krimmer, V. Teague, J. Pomares, A Review of E-Voting: The Past, Present and Future, Annals of Telecommunications 71 (7) (2016) 279–286. doi:10.1007/s12243-016-0525-8.
- [2] D. Bernhard, O. Pereira, B. Warinschi, How Not to Prove Yourself: Pitfalls of the Fiat-Shamir Heuristic and Applications to Helios, in: Advances in Cryptology – ASIACRYPT 2012, Vol. 7658 of Lecture Notes in Computer Science, Springer, Heidelberg, 2012, pp. 626–643.
- [3] Y. Wu, S. Kasahara, Smart Contract-Based E-Voting System Using Homomorphic Encryption and Zero-Knowledge Proof, in: Applied Cryptography and Network Security Workshops (ACNS 2023), Vol. 13907 of Lecture Notes in Computer Science, Springer, Cham, 2023, pp. 67–83. doi:10.1007/978-3-031-41181-6_4.
- [4] V. Farzaliyev, C. Pärn, H. Saarse, J. Willemson, Lattice-Based Zero-Knowledge Proofs in Action: Applications to Electronic Voting, Journal of Cryptology 38 (1) (2025). doi:10.1007/s00145-024-09530-5.
- [5] T. ElGamal, A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms, IEEE Transactions on Information Theory 31 (4) (1985) 469–472. doi:10.1109/TIT.1985.1057074.
- [6] S. Goldwasser, S. Micali, C. Rackoff, The Knowledge Complexity of Interactive Proof Systems, SIAM Journal on Computing 18 (1) (1989) 186–208, conference version: Proc. 17th ACM STOC, 1985, pp. 291–304. doi:10.1137/0218012.
- [7] D. Chaum, T. P. Pedersen, Wallet Databases with Observers, in: Advances in Cryptology – CRYPTO 1992, Vol. 740 of Lecture Notes in Computer Science, Springer, Berlin, Heidelberg, 1992, pp. 89–105.
- [8] Microsoft Corporation, ElectionGuard: Protecting Democratic Elections through Secure, Verifiable Voting, Microsoft On the Issues Blog, available at <https://blogs.microsoft.com/on-the-issues/2019/05/06/protecting-democratic-elections-through-secure-verifiable-voting/> (2019).

- [9] J. Benaloh, M. Naehrig, O. Pereira, D. S. Wallach, ElectionGuard: A Cryptographic Toolkit to Enable Verifiable Elections, in: 33rd USENIX Security Symposium (USENIX Security 24), USENIX Association, Philadelphia, PA, 2024, pp. 5485–5502, ISBN: 978-1-939133-44-1.
- [10] J. Benaloh, M. Naehrig, O. Pereira, ElectionGuard Design Specification, Version 2.1.0, Tech. rep., Microsoft Research, available at <https://github.com/Election-Tech-Initiative/electionguard> (August 2024).
- [11] J. Benaloh, M. Naehrig, O. Pereira, D. S. Wallach, End-to-End Verifiability in Real-World Elections, Technical report, U.S. Election Assistance Commission (EAC), Franklin County, Idaho ElectionGuard Pilot Report (2022).
- [12] G. Peter, A. A. Stonier, A. Sherine, Development of Mobile Application For E-Voting System Using 3-Step Security for Preventing Phishing Attack, in: 2022 2nd International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE), IEEE, 2022, pp. 1–6. doi:10.1109/ICACITE53070.2022.9823791.
- [13] B. Adida, Helios: Web-based Open-Audit Voting, in: Proceedings of the 17th USENIX Security Symposium, USENIX Association, San Jose, CA, USA, 2008, pp. 335–348.
- [14] V. Cortier, P. Gaudry, S. Glondu, Belenios: A Simple Private and Verifiable Electronic Voting System, in: Foundations of Security, Protocols, and Equational Reasoning, Vol. 11565 of Lecture Notes in Computer Science, Springer, Cham, 2019, pp. 214–238.
- [15] E. Zaghloul, T. Li, J. Ren, D-BAME: Distributed Blockchain-Based Anonymous Mobile Electronic Voting, IEEE Internet of Things Journal 8 (22) (2021) 16585–16597. doi:10.1109/JIOT.2021.3051585.
- [16] V. Cortier, C. C. Drăgan, F. Dupressoir, P.-Y. Strub, B. Warinschi, Machine-Checked Proofs for Electronic Voting: Privacy and Verifiability for Belenios, in: 31st IEEE Computer Security Foundations Symposium (CSF 2018), IEEE, 2018, pp. 298–312.
- [17] S. Bell, J. Benaloh, M. D. Byrne, D. DeBeauvoir, B. Eakin, G. Fisher, P. Kortum, N. McBurnett, J. Montoya, M. Morrison-Smith, O. Pereira, P. B. Stark, D. S. Wallach, M. Winn, STAR-Vote: A Secure, Transparent, Auditable, and Reliable Voting System, in: 2013 Electronic Voting Technology Workshop / Workshop on Trustworthy Elections (EVT/WOTE 2013), USENIX Association, Washington, D.C., 2013, available at <https://www.usenix.org/conference/evtvote13/workshop-program/presentation/bell>.
- [18] M. R. Clarkson, S. Chong, A. C. Myers, Civitas: Toward a Secure Voting System, in: 2008 IEEE Symposium on Security and Privacy (SP), IEEE, 2008, pp. 354–368. doi:10.1109/SP.2008.32.
- [19] P. Y. A. Ryan, P. B. Rønne, V. Iovino, Selene: Voting with Transparent Verifiability and Coercion-Mitigation, in: Financial Cryptography and Data Security: FC 2016 Workshops, BITCOIN, VOTING, and WAHC, Vol. 9604 of Lecture Notes in Computer Science, Springer, Heidelberg, 2016, pp. 176–192. doi:10.1007/978-3-662-53357-4_12.
- [20] P. Y. A. Ryan, D. Bismark, J. Heather, S. Schneider, Z. Xia, Prêt à Voter: a Voter-Verifiable Voting System, IEEE Transactions on Information Forensics and Security 4 (4) (2009) 662–673. doi:10.1109/TIFS.2009.2033233.
- [21] K. Gjølsteen, The Norwegian Internet Voting Protocol, in: Towards Trustworthy Elections, Vol. 6000 of Lecture Notes in Computer Science, Springer, 2010, pp. 90–104. doi:10.1007/978-3-642-12980-3_7.
- [22] S. Panja, B. Roy, A Secure End-to-End Verifiable E-Voting System Using Blockchain and Cloud Server, Journal of Information Security and Applications 59 (2021) 102816. doi:10.1016/j.jisa.2021.102816.
- [23] U. Devi, S. Bansal, Secure E-Voting System — A Review, in: Hybrid Intelligent Systems. HIS 2022, Vol. 647 of Lecture Notes in Networks and Systems, Springer, Cham, 2023, pp. 1195–1204. doi:10.1007/978-3-031-27409-1_111.
- [24] V. Farzaliyev, J. Willemson, J. Kääramees, Improved Lattice-Based Mix-Nets for Electronic Voting, IET Information Security 17 (3) (2023) 352–370. doi:10.1049/ise2.12089.
- [25] E. Y. Daraghmi, Y.-H. Daraghmi, S.-M. Daraghmi, Multi-Layered Microservices for Blockchain-Powered E-Voting in IoT-Driven Social Networks: A Distributed Fog and Cloud Personalization Model, Preprint, available at ResearchGate, <https://www.researchgate.net/publication/390763875> (2025).

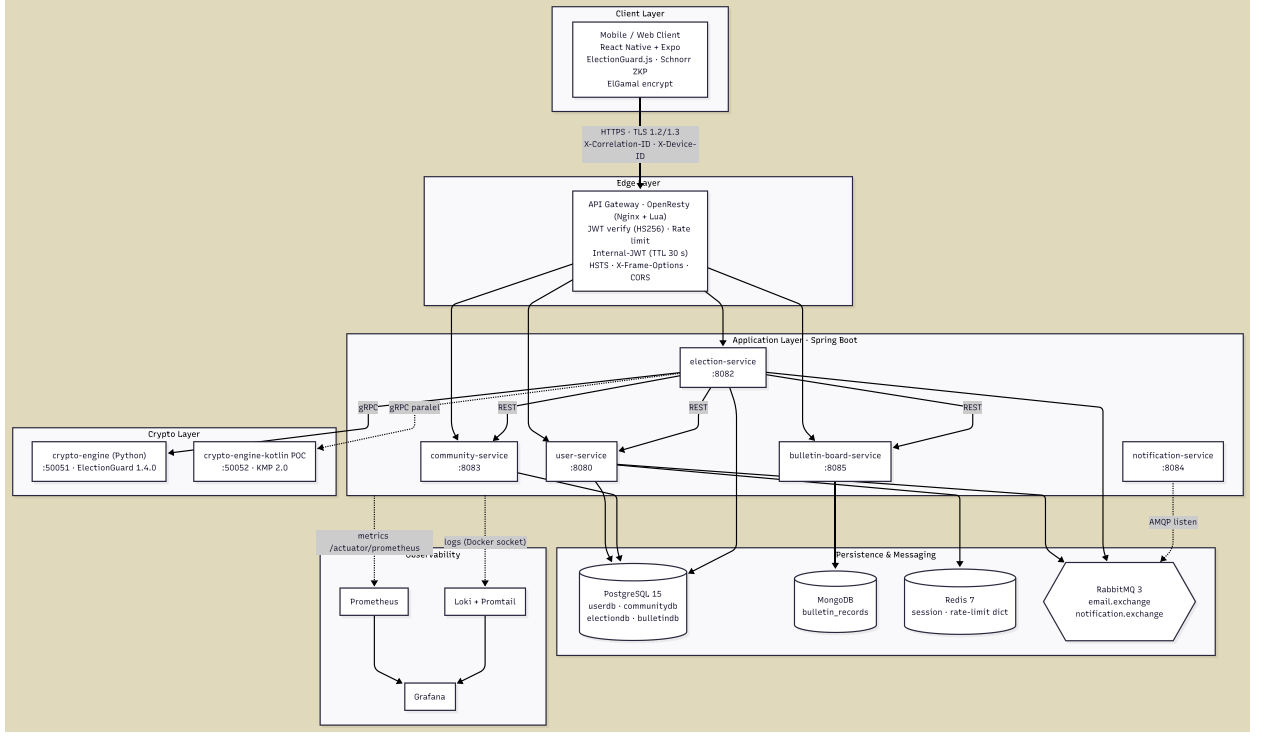


Figure 1: Pallot/Vocket sistem mimarisinin yüksek seviyeli görünümü. Mobil istemciden kalıcılık katmanına (persistence layer) kadar olan veri akışı ve protokol sınırları gösterilmektedir.



Figure 2: Seçim durum makinesi. DRAFT durumundan ARCHIVED durumuna kadar beş aşama ve geçiş koşulları gösterilmektedir.

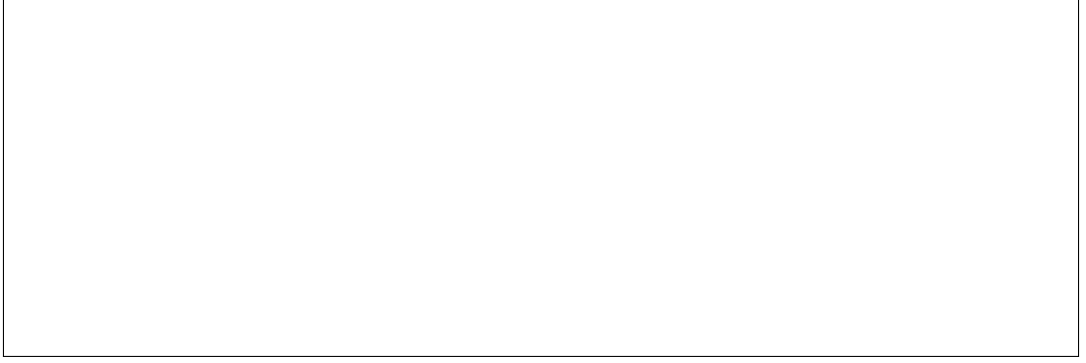


Figure 3: ElectionGuard kriptografik hash zinciri türetimi. `crypto_extended_base_hash`, oy şifreleme ve ZKP doğrulamasında ortak referans noktasını oluşturur.



Figure 4: Uçtan uca oy verme akışı. Anahtar töreni, oy gönderimi ve eşikli sayım aşamaları sırayla gösterilmektedir.

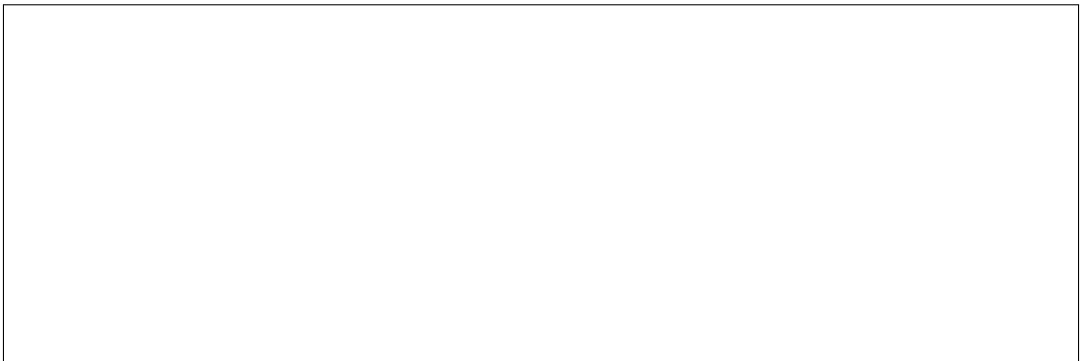


Figure 5: RabbitMQ üzerinden asenkron olay akışı. Direct ve Topic exchange yapıları, üreticiler, tüketiciler ve DLQ'lar gösterilmektedir.

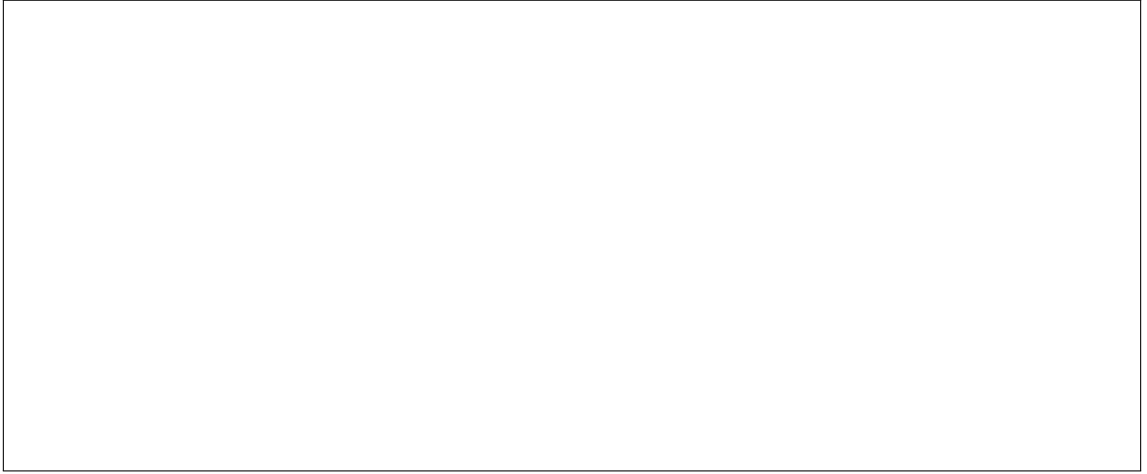


Figure 6: Üretim Docker container topolojisi. Kenar, uygulama, kriptografi, veri ve gözlemlenebilirlik katmanları ile aralarındaki bağımlılıklar gösterilmektedir.